JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JomCode Coding Bootcamp

CHAPTER 16

Supabase authentication and authorization

```
learn / type / run / explain
```

Plain meaning and technical explanation

PLAIN MEANING

Authentication identifies the user. Authorization decides what that user may do.

TECHNICAL EXPLANATION

Supabase issues session tokens. The backend verifies a bearer token and derives the user ID. Ownership checks and RLS enforce access at different layers.

REAL-WORLD USE

Protect writes and allow only the author to edit or delete a post.

QUICK TIP

Never trust an author_id supplied by the browser. Derive it from the verified token.

INSTALL / RUN

Install fastapi, uvicorn, supabase and python-dotenv. Set SUPABASE_URL and SUPABASE_PUBLISHABLE_KEY in .env. Start workbook:app. Use a practice account; do not paste credentials into slides.

Supabase URL and publishable key

Open your practice Supabase project > Connect. Copy its HTTPS Project URL.

Open Settings > API Keys. Copy the publishable key beginning sb_publishable_.

Save both values in .env beside workbook.py, using the exact variable names below.

Legacy note: an anon key serves the older public-key role. Never substitute a private server key for a public key.

```
SUPABASE_URL=https://YOUR_PROJECT.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_REPLACE
```

Official reference

Email authentication and Swagger token flow

Supabase > Authentication > Sign In / Providers > Email: enable email/password sign-in.

Set the development Site URL. Confirm the practice account email if confirmation is enabled.

Start the lab, call /signup and /login, then copy the returned access_token privately.

Swagger Authorize expects the user access token. The publishable application key and user token have different purposes.

```
python -m pip install fastapi uvicorn supabase python-dotenv
python -m uvicorn workbook:app --reload --port 8000
```

Official reference

Worked example 1: shared Python setup (1/2)

workbook.py

```
import os
from uuid import UUID
from typing import Annotated
from dotenv import load_dotenv
from supabase import create_client
from fastapi import FastAPI, Header, HTTPException, Depends
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from pydantic import BaseModel
load_dotenv()
app = FastAPI(title="Chapter 16 authentication lab")
bearer = HTTPBearer(auto_error=False)
def client():
    return create_client(os.environ["SUPABASE_URL"], os.environ["SUPABASE_PUBLISHABLE_KEY"])
class Credentials(BaseModel):
    email: str
```

PRO TIP Keep this setup once at the top of the file.

Worked example 1: shared Python setup (2/2)

workbook.py

```
16        password: str
```

PRO TIP Keep this setup once at the top of the file.

Worked example 1: Register a practice account

workbook.py

```
@app.post("/signup")
def signup(payload: Credentials):
    try:
        result = client().auth.sign_up(payload.model_dump())
        return {"user_id": str(result.user.id),
                "confirmation_required": result.session is None}
    except Exception as error:
        raise HTTPException(400, "Signup failed; inspect Supabase settings") from error
```

PRO TIP Use Supabase dashboard authentication settings to inspect confirmation behavior.

Run and explain example 1

EXPECTED RESULT

A new account returns its user ID. With confirmation enabled, confirmation_required is true.

WHY IT BEHAVES THIS WAY

Creating an account may not create a usable session until the email is confirmed.

JOMHACK
EMPOWERING INNOVATION

Worked example 2: Sign in and receive a token

workbook.py

```
@app.post("/login")
def login(payload: Credentials):
    try:
        session = client().auth.sign_in_with_password(payload.model_dump()).session
        if session is None:
            raise ValueError("No session")
        return {"access_token": session.access_token}
    except Exception as error:
        raise HTTPException(401, "Sign in failed") from error
```

PRO TIP This lab exposes a token for Swagger practice. Do not log or commit it.

Run and explain example 2

EXPECTED RESULT

Valid confirmed credentials return access_token. Invalid credentials return 401.

WHY IT BEHAVES THIS WAY

The access token accompanies later API requests. The password should not be sent on every request.

Worked example 3: Verify the bearer token

workbook.py

```
01    def current_user(credentials: Annotated[
02        HTTPAuthorizationCredentials | None, Depends(bearer)]):
03        if credentials is None:
04            raise HTTPException(401, "Sign in to continue")
05        try:
06            user = client().auth.get_user(credentials.credentials).user
07            return UUID(str(user.id))
08        except Exception as error:
09            raise HTTPException(401, "Invalid or expired session") from error
```

PRO TIP The guided source uses a Header dependency; its Swagger UI has no automatic Authorize button.

Run and explain example 3

EXPECTED RESULT

Swagger Authorize accepts the token. Missing, invalid or expired tokens result in 401.

WHY IT BEHAVES THIS WAY

Verification asks Supabase to validate the token. Merely decoding a token does not establish trust.

Worked example 4: Ownership check

workbook.py

```python
@app.get("/owned/{author_id}")
def owned(author_id: UUID, user_id: UUID = Depends(current_user)):
    if user_id != author_id:
        raise HTTPException(403, "You can only edit your own posts")
    return {"allowed": True}
```

PRO TIP Postlight's trusted database connection can bypass RLS, so backend ownership checks remain essential.

Run and explain example 4

EXPECTED RESULT

Your user ID returns 200. Another valid UUID returns 403. Missing token returns 401.

WHY IT BEHAVES THIS WAY

Authentication runs before the ownership comparison. RLS also protects direct browser database operations.

Exercise 1: Identity endpoint

YOUR TASK

Add GET /me returning only the verified user ID.

CHECK THESE CASES

Valid token gives the authenticated ID. Missing token gives 401.

HINT AFTER A TRY

Reuse Depends(current_user).

Exercise 2: Permission decision

YOUR TASK

Create a pure function for ownership and test anonymous, owner and other-user cases.

CHECK THESE CASES

Anonymous false. Matching IDs true. Different IDs false.

HINT AFTER A TRY

Keep this function independent of the SDK.

Course backend checkpoint 16: integration

01 The four small worked examples establish the concepts. Apply the following complete files to the same course backend folder.

02 This is the course application built through Chapters 14-19, not the official guided project.

03 Each code listing names the file to create or replace. Keep previous files unless a replacement is shown.

04 A complete recovery copy is in workbooks/course_backend/checkpoints/ch16. Configure its .env before starting it.

Course integration: schema_16.sql

course_backend/schema_16.sql

```
-- Chapter 16: extend the same course table; do not use guided_project/schema.sql.
alter table public.jomcode_lesson_posts
 add column if not exists author_id uuid references auth.users(id),
 add column if not exists body text not null default '',
 add column if not exists created_at timestamptz not null default now(),
 add column if not exists updated_at timestamptz not null default now();
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: config.py (1/2)

course_backend/config.py

```
# Course backend configuration. Run from workbooks/course_backend after creating .env.
# Windows: py -m uvicorn main:app --reload. macOS: python3 -m uvicorn main:app --reload.
import os
from dotenv import load_dotenv
from fastapi import HTTPException
load_dotenv()

def required(name, legacy=None):
    value = os.getenv(name) or (os.getenv(legacy) if legacy else None)
    if not value:
        raise HTTPException(503, f"Missing server setting: {name}")
    return value

ORIGINS = os.getenv("FRONTEND_ORIGIN",
    "http://localhost:5173,http://127.0.0.1:5500,http://localhost:5500").split(",")
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: config.py (2/2)

course_backend/config.py

```
16    CHAT_MODEL = os.getenv("GEMINI_CHAT_MODEL", "gemini-3.5-flash-lite")
17    EMBEDDING_MODEL = os.getenv("GEMINI_EMBEDDING_MODEL", "gemini-embedding-2")
18    BUCKET = os.getenv("SUPABASE_STORAGE_BUCKET", "course-images")
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: database.py

course_backend/database.py

```
# Chapter 15 onward: a new server-side database connection for each transaction.
import psycopg
from config import required

def connection():
    return psycopg.connect(required("SUPABASE_DB_URL"))
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: auth.py (1/3)

course_backend/auth.py

```
# Chapter 16: publishable application key plus verified user access token.
# New key names are primary. Legacy aliases are explicit compatibility fallbacks.
from uuid import UUID
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer
from pydantic import BaseModel
from supabase import create_client
from config import required
bearer = HTTPBearer(auto_error=False)

class Credentials(BaseModel):
    email: str
    password: str

def public_client():
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: auth.py (2/3)

course_backend/auth.py

```
16          return create_client(required("SUPABASE_URL"),
17              required("SUPABASE_PUBLISHABLE_KEY", "SUPABASE_ANON_KEY"))
18
19      def current_user(credentials=Depends(bearer)):
20          if credentials is None:
21              raise HTTPException(401, "Sign in to continue")
22          try:
23              user = public_client().auth.get_user(credentials.credentials).user
24              return UUID(str(user.id))
25          except Exception as error:
26              raise HTTPException(401, "Invalid or expired session") from error
27
28      def signup(payload: Credentials):
29          try:
30              result = public_client().auth.sign_up(payload.model_dump())
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: auth.py (3/3)

course_backend/auth.py

```
            return {"user_id": str(result.user.id), "confirmation_required": result.session is None}
        except Exception as error:
            raise HTTPException(400, "Signup failed; inspect the Supabase settings") from error

    def login(payload: Credentials):
        try:
            result = public_client().auth.sign_in_with_password(payload.model_dump())
            if result.session is None:
                raise ValueError("No session")
            return {"access_token": result.session.access_token}
        except Exception as error:
            raise HTTPException(401, "Sign in failed") from error
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: posts.py (1/4)

course_backend/posts.py

```
# Chapter 16: extend the Chapter 15 CRUD lab with verified users and ownership.
from fastapi import HTTPException
from psycopg.rows import dict_row
from database import connection

def validate_text(title, body):
    title, body = title.strip(), body.strip()
    if not 1 <= len(title) <= 160 or not 1 <= len(body) <= 20000:
        raise HTTPException(422, "Invalid title or body length")
    return title, body

def serialize(row):
    return {**row, "author_id": str(row["author_id"]) if row["author_id"] else None,
        "created_at": row["created_at"].isoformat(), "updated_at": row["updated_at"].isoformat()}
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

JOMHACK
EMPOWERING INNOVATION

Course integration: posts.py (2/4)

course_backend/posts.py

```
16    def list_posts():
17        with connection() as conn, conn.cursor(row_factory=dict_row) as cur:
18            cur.execute("select * from public.jomcode_lesson_posts order by id desc")
19            return [serialize(row) for row in cur.fetchall()]
20
21    def get_post(post_id: int):
22        with connection() as conn, conn.cursor(row_factory=dict_row) as cur:
23            cur.execute("select * from public.jomcode_lesson_posts where id=%s", (post_id,))
24            row = cur.fetchone()
25        if row is None:
26            raise HTTPException(404, "Post not found")
27        return serialize(row)
28
29    def save(post_id, user_id, title, body):
30        title, body = validate_text(title, body)
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: posts.py (3/4)

course_backend/posts.py

```
        with connection() as conn, conn.cursor(row_factory=dict_row) as cur:
            if post_id is None:
                cur.execute("""insert into public.jomcode_lesson_posts
                    (author_id,title,body) values (%s,%s,%s) returning *""", (user_id,title,body))
            else:
                cur.execute("select * from public.jomcode_lesson_posts where id=%s for update",
    (post_id,))
                old = cur.fetchone()
                if old is None:
                    raise HTTPException(404, "Post not found")
                if old["author_id"] != user_id:
                    raise HTTPException(403, "Only the author may change this post")
                cur.execute("""update public.jomcode_lesson_posts set title=%s,body=%s,
                    updated_at=now() where id=%s returning *""", (title,body,post_id))
            row = cur.fetchone()
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: posts.py (4/4)

course_backend/posts.py

```
46          return serialize(row)
47
48      def delete_post(post_id, user_id):
49          with connection() as conn, conn.cursor(row_factory=dict_row) as cur:
50              cur.execute("select * from public.jomcode_lesson_posts where id=%s for update", (post_id,))
51              row = cur.fetchone()
52              if row is None:
53                  raise HTTPException(404, "Post not found")
54              if row["author_id"] != user_id:
55                  raise HTTPException(403, "Only the author may delete this post")
56              cur.execute("delete from public.jomcode_lesson_posts where id=%s", (post_id,))
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: main.py (1/3)

course_backend/main.py

```
# Chapter 16 checkpoint. From this folder: python -m uvicorn main:app --reload.
from uuid import UUID
from fastapi import FastAPI, Depends, Response
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from auth import Credentials, current_user, signup, login
from config import ORIGINS
import posts
app = FastAPI(title="Course backend: authentication checkpoint")
app.add_middleware(CORSMiddleware, allow_origins=ORIGINS,
    allow_methods=["GET","POST","PUT","DELETE"], allow_headers=["Authorization","Content-Type"])

@app.get("/health")
def health():
    return {"ok": True}
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Course integration: main.py (2/3)

course_backend/main.py

```
16
17      app.post("/signup")(signup)
18      app.post("/login")(login)
19      app.get("/posts")(posts.list_posts)
20      app.get("/posts/{post_id}")(posts.get_post)
21
22      @app.get("/me")
23      def me(user_id: UUID = Depends(current_user)):
24          return {"id": str(user_id)}
25
26      class PostInput(BaseModel):
27          title: str
28          body: str
29
30      @app.post("/posts", status_code=201)
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

JOMHACK
EMPOWERING INNOVATION

Course integration: main.py (3/3)

course_backend/main.py

```
31    def create(payload: PostInput, user_id: UUID = Depends(current_user)):
32        return posts.save(None, user_id, payload.title, payload.body)
33
34    @app.put("/posts/{post_id}")
35    def update(post_id: int, payload: PostInput, user_id: UUID = Depends(current_user)):
36        return posts.save(post_id, user_id, payload.title, payload.body)
37
38    @app.delete("/posts/{post_id}", status_code=204)
39    def delete(post_id: int, user_id: UUID = Depends(current_user)):
40        posts.delete_post(post_id, user_id)
41        return Response(status_code=204)
```

PRO TIP Create or replace this file only at this chapter checkpoint; retain the other course backend files.

Verify the course checkpoint

Stop the earlier Uvicorn process before starting the current checkpoint on port 8000.

Apply the SQL migrations through Chapter 16. Run from the active course backend folder.

Open /docs and test the features introduced so far. Image writes use multipart forms starting in Chapter 17; earlier Chapter 16 uses JSON.

From Chapter 19 onward, saving a post generates embeddings. Gemini access is required for a successful save.

```
python -m uvicorn main:app --reload --port 8000
```

Checkpoint before moving on

01 Run and explain the four worked examples, including their shared setup.

02 Attempt both exercises before reading their solutions. Check the stated acceptance cases.

03 Explain the real-world connection: Protect writes and allow only the author to edit or delete a post.

Official reference